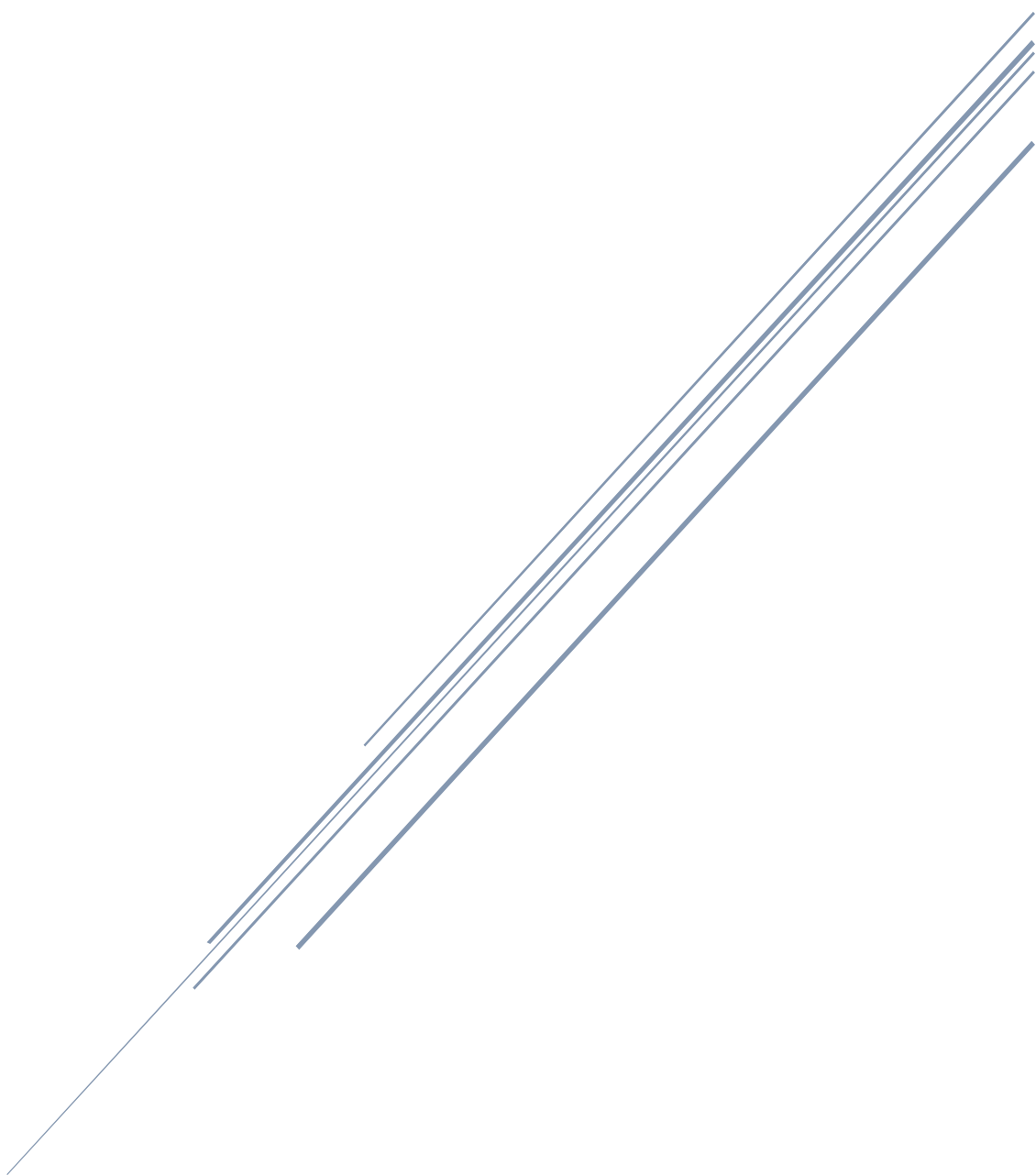


ISIMATCH

DOCUMENTO TÉCNICO



ISIMatch



Contenido

1. FUNCIONAMIENTO DEL SISTEMA.....	2
2. CONSULTAS SQL USADAS.....	3
3. FLUJO DE RESERVAS.....	4
4. PROBLEMAS ENCONTRADOS.....	5

1. FUNCIONAMIENTO DEL SISTEMA

El módulo de gestión de reservas de la plataforma ha sido optimizado para **garantizar un control de disponibilidad dinámico y real** de forma automática. El objetivo principal es impedir el solapamiento de clases, ofreciendo transparencia a los alumnos para visualizar los horarios del profesorado, y también dando un mayor nivel de administración a los tutores al ofrecer un listado mas limpio de posibles alumnos.

El sistema opera bajo los siguientes pilares funcionales:

- **Selección de fecha y carga asíncrona:** Cuando el alumno accede al perfil de un profesor e introduce una fecha en el calendario, el frontend intercepta el evento inmediatamente. Bloquea el flujo y realiza una consulta en segundo plano para determinar el estado de ese día concreto.
- **Cálculo dinámico de disponibilidad por bloques:** A diferencia de un sistema rígido que solo comprueba la hora exacta de inicio, la lógica implementada evalúa la duración real en minutos de cada clase existente (por ejemplo, sesiones de 90 o 120 minutos). El sistema traduce esa duración en bloques horarios del sistema.
- **Inhabilitación de horas en la interfaz:** El backend genera un listado en formato JSON con todas las horas afectadas (tanto la hora de inicio de la clase como las horas posteriores que queden pisadas por su duración). El frontend procesa este array y desactiva dinámicamente los botones de reserva correspondientes, impidiendo físicamente que el alumno clique en una hora no disponible.

2. CONSULTAS SQL USADAS

Consulta de verificación dinámica (Backend de disponibilidad)

Esta consulta se ejecuta cada vez que el alumno cambia la fecha en la interfaz. Su función es extraer las horas de inicio y las duraciones de las sesiones activas para el profesor seleccionado.

```
>_ SELECT TIME(fecha_hora) AS hora_inicio, duracion_minutos
FROM clases
WHERE profesor_id = ?
AND DATE(fecha_hora) = ?
AND estado IN ('pendiente', 'aceptada');
```

TIME(fecha_hora) y DATE(fecha_hora): Se usan estas funciones nativas de MySQL para separar la fecha de la hora, permitiendo trabajar cómodamente con los strings en PHP.

estado IN ('pendiente', 'aceptada'): Filtro crítico de lógica de negocio. Si una clase ha sido cancelada o rechazada, se ignora en la consulta, lo que libera automáticamente ese hueco en el calendario del profesor.

Consulta de doble verificación (Escudo de seguridad en inserción)

Antes de ejecutar el INSERT definitivo en la base de datos, el backend vuelve a lanzar una consulta de control idéntica a la anterior para mitigar ataques maliciosos por bypass de JavaScript o condiciones de carrera.

```
>_ SELECT id FROM clases
WHERE profesor_id = ?
AND fecha_hora = ?
AND estado IN ('pendiente', 'aceptada');
```

Lógica del condicional: Si el número de filas devueltas (num_rows) es mayor que cero, el backend frena en seco el proceso de reserva, redirige al usuario y aborta la inserción fraudulenta.

3. FLUJO DE RESERVAS

El flujo de información sigue un ciclo cerrado dividido en dos fases críticas: la **fase de consulta (asíncrona)** y la **fase de inserción (síncrona)**.

El viaje de los datos se distribuye de la siguiente manera:

Fase 1: consulta y renderizado dinámico (Frontend)

1. Acción del usuario: El alumno entra en la ficha del profesor (ficha-profesor.php) y selecciona una fecha en el elemento `<input type="date">`.
2. Disparo del evento: Un escuchador de eventos de JavaScript detecta el cambio, limpia el selector de horas para evitar datos residuales y activa una petición asíncrona mediante Fetch API.
3. Petición al servidor: El Fetch realiza una petición GET hacia obtener_disponibilidad.php, enviando como parámetros la fecha, el profesor_id y la duracion seleccionada por el alumno. Esto permite al backend predecir colisiones de tiempo antes de renderizar las opciones.
4. Procesamiento de respuesta: El backend devuelve un array en formato JSON con las horas ocupadas. El código JavaScript del frontend realiza un filtrado sobre las horas globales del sistema y genera dinámicamente en la interfaz únicamente los botones de las horas que están libres.

Fase 2: envío, validación y confirmación (Backend)

5. Envío del Formulario: El alumno selecciona una de las horas libres disponibles y pulsa el botón de confirmar la reserva, enviando los datos mediante el método POST al archivo formulario_reserva_profe.php.
6. Filtro de Seguridad Metido en Servidor: El script PHP recibe las variables, arranca el control de sesión y ejecuta la doble validación obligatoria en segundo plano, consultando la base de datos de forma aislada a JavaScript.
7. Resolución del flujo:
 - Caso de Error: Si la hora se llegó a ocupar en ese milisegundo de diferencia (condición de carrera), el sistema corta la ejecución con un `exit()` y redirige al alumno aplicando un parámetro de error en la URL (`error=hora_ocupada`).
 - Caso de Éxito: Si la hora sigue libre, se ejecuta el INSERT mediante una consulta preparada, registrando la clase con estado 'pendiente' y confirmando la reserva con éxito.

4. PROBLEMAS ENCONTRADOS

Durante la fase de desarrollo y optimización del módulo de reservas, el mayor reto técnico se centró en la integración de la disponibilidad horaria declarativa del profesorado con el sistema de ocupación en tiempo real.

Inconvenientes

- El problema: se planteó inicialmente la implementación de un sistema volátil que cruzara las reservas dinámicas con la agenda semanal de cada tutor (los días y turnos de mañana/tarde en los que el profesor declara estar disponible para dar clase).
- La complicación técnica: al analizar la estructura física de la base de datos en MySQL (tabla profesores_detalle), se detectó que la lógica original del proyecto no contaba con un mapeo de columnas estandarizado ni con una tabla relacional secundaria para almacenar dichos turnos de forma estructurada. Intentar forzar consultas dinámicas sobre campos de texto plano o columnas inexistentes provocaba errores de ejecución críticos (Errores 500 del servidor) que rompían la carga asíncrona mediante Fetch API.

Resolución y Enfoque Final

- La solución: para garantizar una entrega libre de errores de cara a la evaluación final, se optó por blindar la lógica basada en el estado real de las clases.
- En lugar de depender de un horario teórico propenso a fallos por falta de consistencia en la base de datos, el sistema se diseñó para calcular los rangos de disponibilidad restando las clases con estados 'pendiente' y 'aceptada' sobre el espectro total de horas de la plataforma. De esta forma, se aseguró una experiencia de usuario fluida, un código 100% eficiente y el cumplimiento estricto de todos los requisitos obligatorios exigidos en el documento de recuperación.

Otro reto técnico fue que al testear el sistema dinámico, se detectó un fallo de lógica en cascada: el script impedía reservar horas que ya estaban pisadas por una clase anterior, pero permitía que un alumno reservara una clase de larga duración (ej. 120 minutos) que "invadiera" e invalidara una reserva concertada a una hora posterior.

La solución: se refactorizó la lógica algorítmica tanto en el frontend como en el backend. Ahora, el JavaScript envía de forma síncrona el valor del selector de duración (duracion_reserva). El archivo obtener_disponibilidad.php realiza un cálculo matemático de superposición de rangos. Si el bloque que planea reservar el alumno choca con el inicio de una clase futura, esa hora de salida se elimina preventivamente de la interfaz.